

Conjunto de Instruções de Máquina

- **Instrução de máquina:**
 - Instrução implementada pelo hardware e visível ao programador
 - Representa comando que é interpretado e executado pelo processador
 - Codificada como sequência de bits
- **Conjunto de instruções de máquina:**
 - ISA (Instruction Set Architecture)
 - Interface entre hardware e software
 - Pode ter diferentes implementações, com custos e desempenhos diferentes
- **Exemplo:** Intel Core i7 e AMD Opteron
 - 2 implementações diferentes do mesmo ISA x86
 - Diferentes pipelines e diferentes organizações de caches

Projeto do Conjunto de Instruções de Máquina

- **Inclui definição de:**
 - Classificação do conjunto de instruções
 - Endereçamento de memória
 - Modos de endereçamento de operandos
 - Tipos e tamanhos dos operandos das instruções
 - Operações das instruções
 - Instruções de alteração do fluxo de controle
 - Codificação das instruções de máquina
- **Fatores conflitantes** $\Rightarrow$ Compromisso (trade-off)

Classificação do ISA

- **Com registradores de propósito específico:**

- Possui vários registradores, porém com restrições de uso
- Pode possuir operandos implícitos: registradores específicos

- **Com conjunto de registradores de propósito geral**

(**GPR** – *General Purpose Registers*):

Exemplo: `c = a + b ;`

- Possui apenas operandos explícitos:
 - Registradores ou posições de memória
- **GPR Load-Store ou Register-Register:**
 - Apenas instruções `load` e `store` acessam memória
 - Instruções aritméticas acessam apenas registradores
 - **Exemplos:** ARM, MIPS, PowerPC, SPARC, RISC-V

```
load R1, a
load R2, b
add R3, R1, R2
store R3, c
```

- **GPR Register-Memory:**

- Qualquer instrução pode acessar memória
- **Exemplos:** Intel x86, IBM 360/370, Motorola 68000

```
load R1, a
add R1, b
store R1, c
```

- **Vantagens e desvantagens?**

Modos de Endereçamento de Operandos

- **Instruções especificam operandos:**
 - Constante, em registrador, em posição de memória
- **Modos de endereçamento:**
 - Determinam como instruções especificam onde está operando
- **Quais modos de endereçamento implementar ?**
 - Por registrador, imediato, direto, indireto por registrador, por deslocamento, indireto pela memória, com auto-incremento/auto-decremento, indexado, ...
- **Exemplos:**
 - **Por deslocamento:** MIPS

```
lw dest, desloc (base)
```

```
# Reg[dest] = Mem[Reg[base] + desloc]
```
 - **Based with scaled indexed and displacement:** Intel x86

```
# Reg[dest] = Mem[Reg[base] + Reg[index] * 2^scale + desloc]
```
- **Vantagens e desvantagens?**

Operações e Operandos

- **Quais instruções implementar ?**

- Lógicas e aritméticas
- Transferência de dados
- **Alteração do fluxo de controle:**
 - Desvios condicionais:
 - Quais testes implementar ?
 - Desvios incondicionais
 - Chamada e retorno de subrotina
- Chamadas ao sistema operacional
- Ponto-flutuante
- Manipulação de strings
- Manipulação gráfica
- ...

- **Que tipos de dados instruções operam ?**

- Inteiro, ponto flutuante (precisão simples e dupla), caracter, ...
- Como instrução indica tipo dos operandos?

Modos de Endereçamento para Instruções de Alteração do Fluxo de Controle

- **Endereçamento direto:**

- Endereço alvo conhecido em tempo de compilação

- **Exemplo:** Intel x86

```
jmp address    # EIP = address
```

- **Endereçamento indireto por registrador:**

- Endereço alvo não precisa ser conhecido em tempo de compilação

- **Exemplo:** MIPS

```
jr reg    # PC = Reg[reg]
```

- **Endereçamento relativo a PC:**

- Endereço alvo conhecido em tempo de compilação

- **Exemplo:** MIPS

```
beq reg1, reg2, desloc    # Se ... então PC = PC + 4 + (desloc << 2)
```

- **Vantagens e desvantagens?**

Desvios Condicionais: Especificação da Condição de Desvio

- **Técnica 1: Registrador de condição**

- Testa registrador qualquer com o resultado de uma comparação

- **Exemplo:** MIPS

```
slt r1, r2, r3      # Se Reg[r2] < Reg[r3] então Reg[r1] = 1 senão Reg[r1] = 0
bne r1, $0, fim     # Se Reg[r1] != 0 (Reg[r2] < Reg[r3]) então desvia
```

- **Técnica 2: Código de condição (CC)**

- Testa bits especiais setados por operações lógico-aritméticas

- **Exemplo:** Intel x86

```
sub r2, r3          # Reg[r2] = Reg[r2] - Reg[r3]
js  fim             # Se flag sign = 1 (Reg[r2] < Reg[r3]) então desvia
```

- **Técnica 3: Compara e desvia**

- Comparação é parte da instrução de desvio

- **Exemplo:** PA-RISC

```
COMBT, < r2, r3, fim # Se Reg[r2] < Reg[r3] então desvia
                    # (compare and branch if true)
```

- **Vantagens e desvantagens?**

Codificação do Conjunto de Instruções

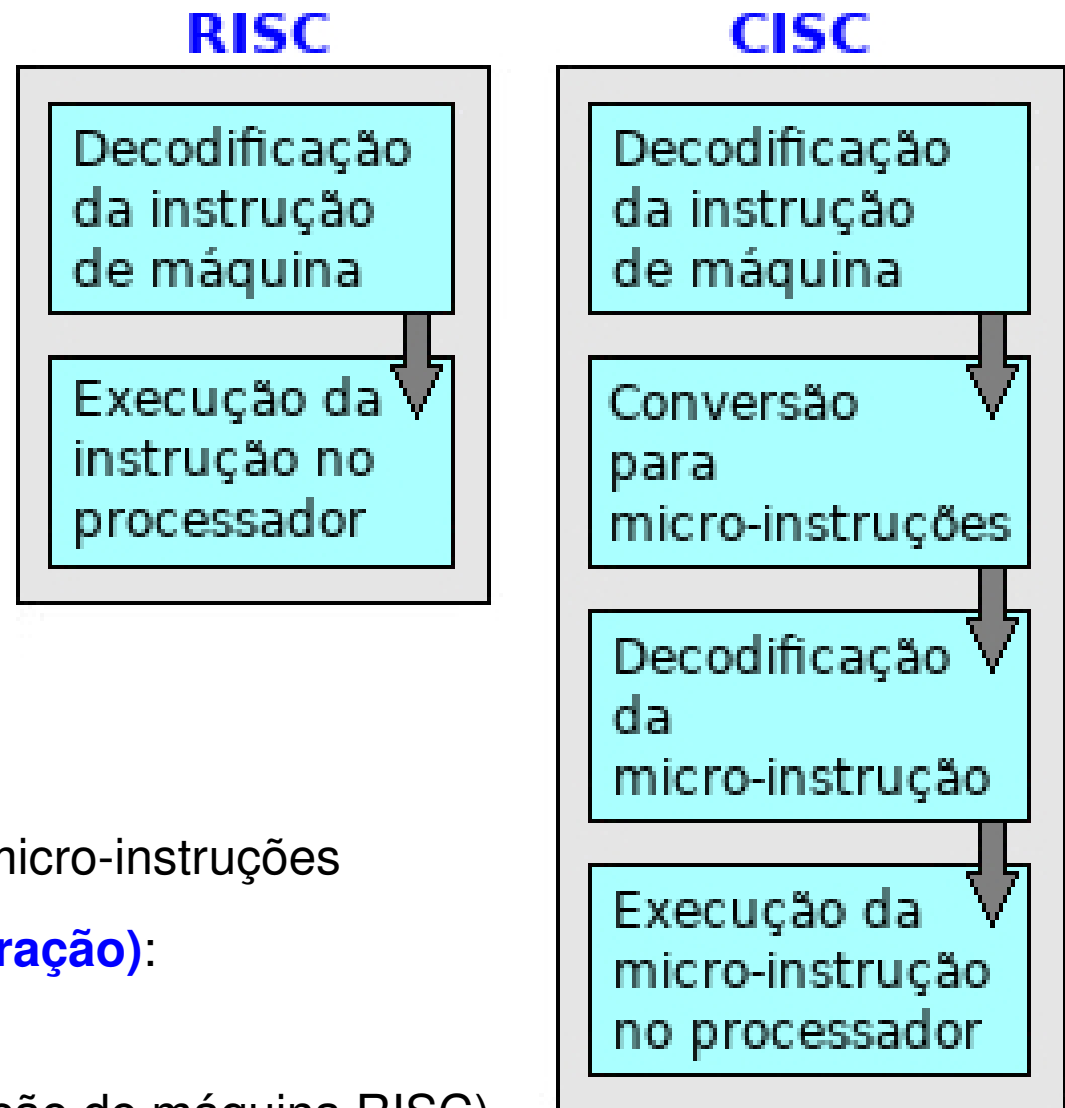
- **Como instruções são codificadas em representação binária ?**
 - Tamanhos das instruções, formatos, campos ?
- **Codificação fixa:**
 - Em geral, todas as instruções com o mesmo tamanho e poucos formatos
 - Modo de endereçamento codificado no *opcode*, juntamente com operação
 - **Exemplos:** MIPS, ARM, PowerPC, SPARC
- **Codificação variável:**
 - Instruções podem ter diferentes tamanhos
 - Permite quase todos os modos de endereçamento para todas as operações
 - Modo de endereçamento codificado em campo para cada operando
 - **Exemplo:** Intel x86
- **Codificação híbrida:**
 - Reduz variação no tamanho e formatos das instruções
 - **Exemplos:** MIPS16, ARM Thumb
- **Vantagens e desvantagens?**

Antigo Debate: Processadores RISC e CISC

- **RISC**: Reduced Instruction Set Computer
- **CISC**: Complex Instruction Set Computer

	RISC	CISC
Arquitetura	load-store	register-memory (em geral)
Instruções	poucas e simples	muitas, simples e complexas
Modos de endereçamento	poucos e simples	muitos, simples e elaborados
Codificação das instruções	fixa	variável
Tamanho das instruções	único tamanho	diversos tamanhos
Formato das instruções	poucos formatos	diversos formatos
Decodificação da instrução	simples	complexa
Unidade de controle	simples	microprogramada
Desempenho	aumenta n ^o de instruções reduz CPI	reduz n ^o de instruções aumenta CPI

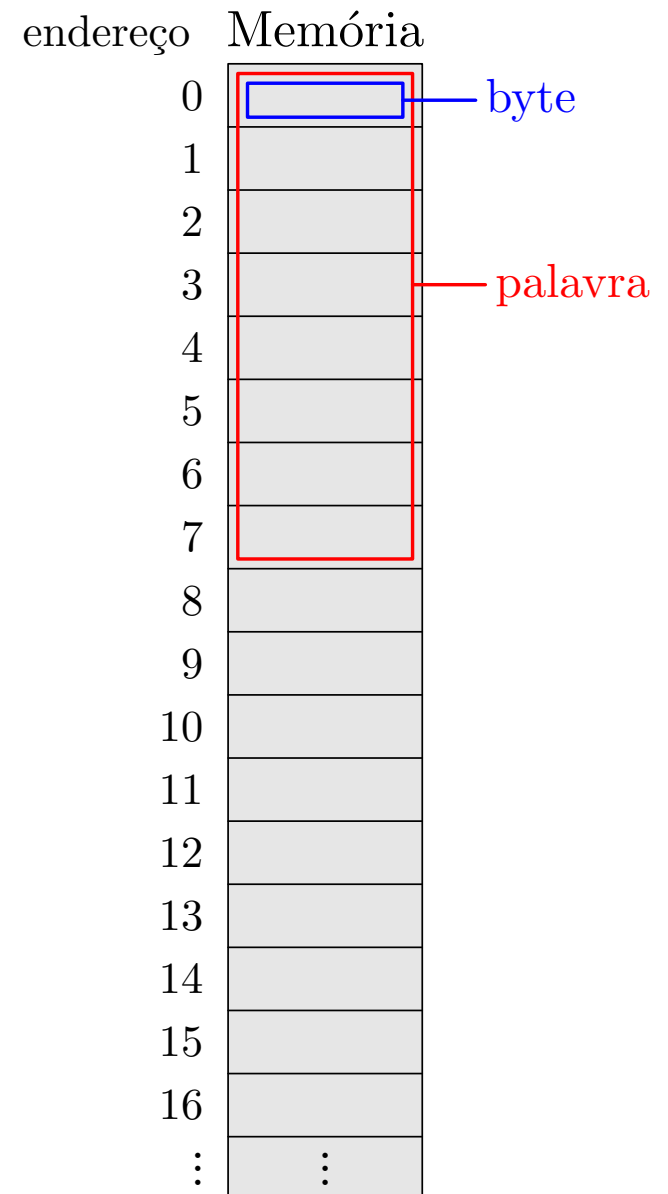
Processadores RISC e CISC: Execução das Instruções



- **Instrução de máquina CISC:**
 - Implementada como sequência de micro-instruções
- **Cada micro-instrução (ou micro-operação):**
 - Leva 1 ciclo do clock
 - Instrução simples (como uma instrução de máquina RISC)
 - Corresponde a um passo da execução de uma instrução de máquina CISC
- **Microprograma (ou microcódigo):**
 - Conjunto de micro-instruções que implementam o conjunto de instruções de máquina CISC

Conjunto de Instruções MIPS64

- **Palavra:** de 64 bits (8 bytes)
- **Conjunto de registradores** de inteiros de propósito geral: GPR
 - 32 registradores de 64 bits: R0, R1, ..., R31
 - R0 tem valor constante 0
- **Conjunto de registradores** de ponto flutuante: FPR
 - 32 registradores de 64 bits: F0, F1, ..., F31
- **Registrador PC:** Program Counter
- **Memória:**
 - Endereçada por bytes
 - Endereços de memória de 64 bits
 - Restrição de alinhamento de palavras
- **Instruções de tamanho fixo:** 32 bits
- **3 formatos de instruções:** R, I, J
- **Modos de endereçamento para dados:**
 - Imediato, por registrador, por deslocamento
- **Modos de endereçamento para desvios:**
 - Relativo a PC, pseudo-direto, indireto por registrador



Instruções MIPS64

Transferências de dados: load e store

Load double word	LD R1, 30 (R2)
Store double word	SD R1, 30 (R2)
Load FP double	L.D F1, 30 (R2)
Store FP double	S.D F1, 30 (R2)

Lógico-aritméticas

Add	DADD R1, R2, R3
Add immediate	DADDI R1, R2, 15
Sub	DSUB R1, R2, R3
And	DAND R1, R2, R3
Or	DOR R1, R2, R3
Add FP	ADD.D F1, F2, F3
Sub FP	SUB.D F1, F2, F3
Mul FP	MUL.D F1, F2, F3
Div FP	DIV.D F1, F2, F3

Controle e desvios condicionais e incondicionais

BEQ	BEQ R1, R2, ...
BNE	BNE R1, R2, ...
J	J ...
BEQZ	BEQZ R1, ... Se Reg[R1] == 0 então desvia
MOVZ	MOVZ R1, R2, R3 Se Reg[R3] == 0 então Reg[R1] = Reg[R2]